

SIMATS ENGINEERING

ASSESSMENT TOOL 2: Research Review & Analysis

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS COURSE CODE: MLA01

COURSE OUTCOME COVERED

CO4: *Develop intelligent software agents using suitable agent architectures and learning techniques.* (BTL 3)

Assessment Tool Weightage: 30%

Total Marks: 100

Time: 90 Mins

No. of Questions: 1

Annexure A

Research Review & Analysis

Code of Conduct:

I Shaik Mubarak(192425194) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Literature Search and Source Selection	10%	
3. Literature Review and Synthesis	15%	

4. Comparative Analysis of AI Techniques	10%	
5. Critical Analysis of Research Findings	10%	
6. Research Gap Identification	10%	
7. Proposed Research Direction	10%	
8. Research Methodology / Analysis Framework	5%	
9. Experimental / Evidence-Based Validation	10%	
10. Result Analysis and Interpretation	5%	
11. Research Paper Quality and Referencing	10%	
12. Technical Presentation and Research Defense	5%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

ANSWER ANY ONE QUESTION

S.No.	Research Review & Analysis Question	Primary Topic
1	Decision Tree Learning: Review recent research on ID3, C4.5, and CART. Compare their attribute-selection methods, pruning strategies, interpretability, computational complexity, and classification performance. Identify a research gap and suggest a possible improvement.	Decision Trees
2	Neural Networks vs Statistical Learning: Review and compare research on neural-network and statistical-learning approaches for prediction and classification. Analyze their performance, data requirements, generalization, interpretability, computational cost, and suitability for different applications.	Statistical Learning & Neural Networks
3	Kernel Methods: Analyze research on SVM and kernel methods for non-linear classification. Compare linear, polynomial, and RBF kernels and evaluate their advantages and limitations compared with neural-network-based approaches.	Kernel Methods
4	Reinforcement Learning: Review research on Q-Learning, SARSA, and Deep Q-Networks. Compare their learning mechanisms, exploration strategies, convergence, computational requirements, and real-world applications. Identify challenges and research gaps.	Reinforcement Learning

5	Explainable and Generalizable AI: Review research on Inductive Learning and Explanation-Based Learning and analyze how these approaches contribute to generalization and explainability in AI. Identify current limitations and propose future research directions.	Inductive & Explanation-Based Learning
---	---	--

Structure of the Report:

S.No.	Paper Section	Expected Content
1	Title	Specific and concise title related to the selected research question/topic.
2	Abstract	Brief summary of the research problem, review methodology, major findings, research gap, and conclusion.
3	Keywords	4–6 important keywords related to the selected topic.
4	1. Introduction	Introduce the AI topic, background, importance, motivation, and scope of the review.
5	2. Research Problem and Objectives	Clearly state the research problem and specific objectives of the review.
6	3. Literature Review	Review relevant research papers and summarize major approaches, methodologies, findings, and contributions.
7	4. Comparative Analysis	Compare the reviewed approaches based on suitable parameters such as accuracy, complexity, interpretability, computational cost, scalability, and generalization.
8	5. Critical Analysis	Discuss strengths, weaknesses, limitations, assumptions, and practical challenges identified across the reviewed studies.
9	6. Research Gap	Identify unresolved problems, limitations in existing research, and areas requiring further investigation.

10	7. Proposed Research Direction	Suggest possible solutions, improvements, or future research directions based on the identified gap.
11	8. Discussion	Discuss the overall insights obtained from the review and their implications for AI research and applications.
12	9. Conclusion	Summarize the major findings, key insights, research gap, and future direction.
13	References	List all research papers, books, datasets, websites, and other sources using a consistent citation style.

Evaluation Rubrics:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Research Problem Identification and Understanding	10%	9–10% – Clearly identifies the research problem, objectives, scope, and relevance to AI learning approaches.	7–8% – Research problem and objectives are clearly identified with minor omissions.	5–6% – Basic understanding of the research problem with some gaps.	0–4% – Problem is unclear, inappropriate, or poorly understood.
2. Literature Search and Source Selection	10%	9–10% – Uses diverse, highly relevant, recent, and credible research papers, journals, conferences, and technical sources.	7–8% – Uses relevant and credible sources with minor limitations.	5–6% – Uses a limited number of relevant sources.	0–4% – Sources are insufficient, unreliable, or largely irrelevant.

3. Literature Review and Synthesis	15%	13–15% – Provides a comprehensive review and synthesizes findings across multiple studies rather than merely summarizing them.	10–12% – Good review with meaningful comparison of major studies.	7–9% – Basic review with limited synthesis and comparison.	0–6% – Superficial review with little understanding of existing research.
4. Analysis of AI Techniques / Methods	15%	13–15% – Provides technically accurate and in-depth analysis of relevant methods such as decision trees, statistical learning, neural networks, kernel methods, or reinforcement learning.	10–12% – Provides good technical analysis with minor gaps.	7–9% – Provides basic technical description with limited analysis.	0–6% – Methods are poorly understood or inaccurately described.
5. Comparative Analysis of Research Approaches	10%	9–10% – Critically compares approaches based on accuracy, complexity, interpretability, scalability, computational cost, and application suitability.	7–8% – Provides meaningful comparison using several parameters.	5–6% – Basic comparison with limited criteria.	0–4% – Little or no meaningful comparison.

6. Critical Evaluation of Research Findings	10%	9–10% – Critically evaluates strengths, weaknesses, assumptions, experimental methods, and findings of existing studies.	7–8% – Provides good critical evaluation with minor gaps.	5–6% – Some critical observations are provided but analysis is limited.	0–4% – Mostly descriptive with little critical evaluation.
7. Identification of Research Gaps	10%	9–10% – Clearly identifies significant research gaps, limitations, unresolved issues, and opportunities for further investigation.	7–8% – Identifies relevant research gaps with reasonable justification.	5–6% – Identifies basic gaps but provides limited justification.	0–4% – Research gaps are missing or incorrectly identified.
8. Research Insights and Recommendations	5%	5% – Provides insightful conclusions and technically feasible recommendations for future research or applications.	4% – Provides relevant conclusions and recommendations.	2–3% – Basic conclusions with limited recommendations.	0–1% – Conclusions or recommendations are absent or inappropriate.
9. Technical Report and Referencing	10%	9–10% – Report is well structured, technically accurate, professionally written, and uses a consistent citation/reference style.	7–8% – Well-structured report with minor writing or referencing issues.	5–6% – Adequate report but contains several structural or referencing issues.	0–4% – Poorly structured, inadequately referenced, or incomplete report.

10. Originality, Academic Integrity and Use of Sources	5%	5% – Demonstrates original synthesis, proper citation, and strong academic integrity with no inappropriate copying.	4% – Good originality and proper citation with minor issues.	2–3% – Some original analysis but significant dependence on sources.	0–1% – Evidence of substantial copying, poor citation, or academic-integrity concerns.
--	----	---	--	--	--

Topic 4: Reinforcement Learning

Q-Learning, SARSA, and Deep Q-Networks (DQN)

Abstract

Reinforcement learning (RL) is a learning paradigm in which an agent interacts with an environment, receives rewards, and improves its action-selection strategy to maximize cumulative return. This review focuses on three important value-based approaches: Q-Learning, SARSA, and Deep Q-Networks (DQN). Q-Learning is an off-policy temporal-difference method that updates values toward the best estimated next action, whereas SARSA is an on-policy method that updates using the action actually selected by the current policy. DQN extends Q-Learning to large or high-dimensional state spaces by replacing the Q-table with a neural-network function approximator and using experience replay and a target network to improve training stability. The approaches are compared using specific parameters including learning mechanism, policy type, state representation, exploration strategy, convergence behavior, sample efficiency, memory requirement, computational cost, and application suitability. The review identifies a practical gap: no single basic algorithm provides the simplicity and low computation of tabular methods while also providing the scalability and representation learning ability of DQN. A practical research direction is therefore to use adaptive exploration and prioritized experience selection, with careful reward design and stability controls. The final section provides an implementation framework that evaluates Q-Learning, SARSA, and DQN in a common CartPole environment.

Keywords

Reinforcement Learning, Q-Learning, SARSA, Deep Q-Network, Exploration–Exploitation, Temporal-Difference Learning

1. Introduction

Reinforcement learning differs from supervised learning because the agent is not given a correct label for every input. Instead, it learns by interacting with an environment and observing rewards. At each time step, the agent observes a state, selects an action according to a policy, receives a reward, and moves to a new state. The objective is to maximize the expected discounted cumulative

reward. The assessment question requires a review of Q-Learning, SARSA, and Deep Q-Networks and a comparison of their learning mechanisms, exploration strategies, convergence, computational requirements, and real-world applications. This report follows that structure. Recent research continues to study the stability, reward design, exploration strategy, and sampling efficiency of SARSA and DQN. For example, a 2025 study reported improvements to SARSA and DQN through changes to the epsilon-greedy strategy, reward function, value iteration, and experience sampling, with experiments on path planning, CartPole, and MountainCar. [cite turn0search0](#)

2. Research Problem and Objectives

2.1 Research Problem

The central research problem is to determine how Q-Learning, SARSA, and DQN differ in learning behavior, exploration, convergence, computational requirements, and application suitability, and to identify a research gap that can motivate an improved reinforcement-learning approach.

2.2 Objectives

- Review the learning mechanisms of Q-Learning, SARSA, and DQN.
- Compare the algorithms using specific technical parameters.
- Analyze exploration and exploitation strategies and their effect on learning.
- Compare convergence characteristics, sample efficiency, memory requirements, and computational cost.
- Identify strengths, weaknesses, and practical limitations.
- Identify a research gap related to exploration, reward design, sampling, or scalability.
- Propose a feasible research direction and provide an implementation framework for experimental validation.

3. Literature Review

3.1 Q-Learning

Q-Learning learns an action-value function $Q(s,a)$, representing the expected long-term return for taking action a in state s . It is an off-policy temporal-difference method because the update targets the best estimated next action even when the behavior policy uses exploration. The standard update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Here, α is the learning rate and γ is the discount factor. During training, epsilon-greedy exploration is commonly used: with probability ϵ the agent explores by choosing a random action; otherwise it selects the action with the highest Q-value. Q-Learning is simple and effective for small discrete state-action spaces, but a table becomes impractical when the state space is large.

3.2 SARSA

SARSA stands for State-Action-Reward-State-Action. It is an on-policy temporal-difference method because the update uses the next action actually selected by the current behavior policy. Its update is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma Q(s',a') - Q(s,a)]$$

The use of the actual next action makes SARSA sensitive to the behavior policy and its exploration level. This can make SARSA more conservative than Q-Learning in risky environments. Research comparing Q-Learning and SARSA has reported this difference in practical settings; for example, a 2024 study of cloud autoscaling found SARSA useful for reducing execution time and monetary cost in its experimental setting. [\[cite turn0search6\]](#)

3.3 Deep Q-Network (DQN)

DQN combines Q-Learning with a deep neural network that approximates $Q(s,a)$. This avoids maintaining a separate table entry for every possible state-action pair and makes the method suitable for larger state representations. DQN typically uses two important stabilization mechanisms: experience replay and a target network. Experience replay stores transitions and trains on randomly sampled mini-batches, reducing strong temporal correlations. The target network is updated less frequently so that the target used for learning does not change at every training step. DQN also commonly uses epsilon-greedy exploration. A recent control-oriented survey describes DQN as a prominent deep RL method and emphasizes the role of replay buffers and target networks in stabilizing neural-network-based value learning. [\[cite turn0search3\]](#)

3.4 Research Findings from Recent Studies

Recent literature indicates that RL performance depends not only on the core update equation but also on exploration, reward design, sampling, state representation, and the environment. A 2025 study specifically improved SARSA and DQN through four areas—epsilon-greedy policy, reward function, value iteration, and sampling probability—and reported improved convergence, reward, and training stability in path planning, CartPole, and MountainCar. [\[cite turn0search0\]](#)

A 2026 review also identifies high-dimensional decision-making, sparse or delayed feedback, parameter sensitivity, and long-term credit assignment as continuing challenges for reinforcement learning. It discusses safe and constrained RL, offline RL, world models, and other directions as ways to improve real-world deployability. [cite](#) [turn0search1](#)

4. Comparative Analysis

The following comparison uses specific technical parameters so that the three algorithms can be evaluated consistently rather than only described conceptually.

Parameter	Q-Learning	SARSA	DQN
Learning type	Model-free, value-based	Model-free, value-based	Model-free, deep value-based
Policy	Off-policy	On-policy	Off-policy
State representation	Q-table	Q-table	Neural-network input
Action-value update	Uses max next Q-value	Uses Q-value of actual next action	Neural-network Bellman target
Exploration	Usually ϵ -greedy	Usually ϵ -greedy	Usually ϵ -greedy
Large/continuous state spaces	Poor with plain table	Poor with plain table	Much more suitable
Memory	Low for small tables; grows with state-action space	Low for small tables; grows with state-action space	Replay buffer + neural-network parameters
Computation	Low per update	Low per update	High because of neural-network training
Sample efficiency	Good in small discrete spaces	Good in small discrete spaces	Can require many interactions
Stability	Usually stable in tabular settings	Usually stable in tabular settings	Sensitive to hyperparameters; replay/target network improve stability
Interpretability	High; Q-table can be inspected	High; Q-table can be inspected	Lower; neural network is less transparent
Typical applications	Grid worlds, routing, scheduling	Risk-sensitive navigation, scheduling, control	Games, control, visual/high-dimensional tasks

4.1 Specific Parameters for Experimental Comparison

For a fair student-level experiment, the three algorithms should be tested in the same environment and evaluated with the same episode budget. The following parameters are concrete

starting values. They are experimental settings, not universal optimal values; changing them is part of hyperparameter analysis.

Parameter	Q-Learning	SARSA	DQN
Environment	CartPole-v1	CartPole-v1	CartPole-v1
Episodes	500	500	500
Max steps/episode	500	500	500
Learning rate α	0.10	0.10	0.001
Discount γ	0.99	0.99	0.99
Initial ϵ	1.00	1.00	1.00
Minimum ϵ	0.01	0.01	0.05
ϵ decay	0.995	0.995	0.995
Q-table	20×20×20×20×2 after discretization	20×20×20×20×2 after discretization	Not used
Replay buffer	Not used	Not used	50,000
Batch size	Not used	Not used	64
Target update	Not used	Not used	Every 10 episodes
Optimizer	Not used	Not used	Adam
Hidden layers	Not used	Not used	128 → 128
Loss	TD error	TD error	Mean squared error / Huber-style TD loss

Important limitation: Q-Learning and SARSA in this implementation use discretization because their tabular form cannot directly represent the continuous CartPole observation vector. DQN receives the original four-dimensional observation vector. Therefore, the comparison is useful for illustrating the algorithmic trade-off, but it is not a perfectly matched representation comparison. This limitation should be stated in the report instead of pretending that the three implementations have identical input representations.

5. Critical Analysis

5.1 Q-Learning – Strengths and Weaknesses

- Strength: simple update rule and easy implementation.
- Strength: high interpretability because the learned Q-table can be inspected.
- Weakness: table size grows rapidly with the number of states and actions.
- Weakness: off-policy maximization can produce aggressive behavior in risky environments.

5.2 SARSA – Strengths and Weaknesses

- Strength: on-policy learning incorporates the consequences of the actual exploratory action.

- Strength: can learn more conservative behavior in environments where exploratory mistakes are costly.
- Weakness: performance depends strongly on the behavior policy and exploration schedule.
- Weakness: tabular SARSA has the same scalability problem as tabular Q-Learning.

5.3 DQN – Strengths and Weaknesses

- Strength: neural networks can generalize across large state spaces.
- Strength: suitable for complex observations where a Q-table is impractical.
- Weakness: higher computational and memory requirements.
- Weakness: training can be unstable and sensitive to learning rate, reward scale, exploration schedule, replay strategy, and network architecture.

6. Research Gap

The literature shows that basic Q-Learning and SARSA are attractive because of their simplicity, while DQN provides scalability through function approximation. However, a direct practical tension remains: simple tabular methods are difficult to scale, while DQN introduces substantial computational cost and additional hyperparameters. Recent work has improved SARSA and DQN by modifying exploration, rewards, value iteration, and sampling, demonstrating that these components are important determinants of convergence and stability. [\[cite turn0search0\]](#)

A specific research gap suitable for this project is the lack of a lightweight adaptive strategy that dynamically adjusts exploration and experience selection according to learning progress while preserving a simple experimental design. In other words, instead of using one fixed epsilon-decay schedule throughout training, the agent could adapt exploration based on reward improvement, TD-error statistics, or recent performance.

7. Proposed Research Direction

The proposed direction is an Adaptive Exploration and Sampling Reinforcement Learning approach. The baseline algorithms remain Q-Learning, SARSA, and DQN, but the experiment adds adaptive control of exploration and, for DQN, prioritized or performance-aware sampling. The purpose is not to claim a new state-of-the-art algorithm; it is to create a testable improvement that can be evaluated using measurable outcomes.

7.1 Proposed Specific Parameters

Parameter	Baseline	Proposed setting
Exploration ϵ	Fixed exponential/linear decay	Adaptive: reduce ϵ when reward improves; temporarily increase when performance stagnates
Reward signal	Environment reward	Environment reward + optional bounded shaping for progress
DQN replay	Uniform random sampling	Priority based on TD-error magnitude
Evaluation	Average reward	Average reward + success/solved rate + convergence episode
Stability	Single training run	At least 5 random seeds for mean $\pm$ standard deviation

Two concrete examples of the research direction are: (1) if the moving-average reward stops improving for a fixed number of episodes, temporarily increase ϵ to encourage exploration; and (2) in DQN, sample transitions with larger TD errors more frequently while preventing a few transitions from dominating the buffer. These changes directly address exploration and sampling issues highlighted in recent research. □cite□turn0search0□

8. Discussion

The comparison shows that no algorithm is universally best. Q-Learning is appropriate when the state-action space is small and interpretability is important. SARSA is attractive when the actual behavior policy matters and conservative learning is useful. DQN becomes more appropriate as the state representation becomes large and function approximation is required.

The most important practical issue is that algorithm selection cannot be separated from the environment. A method that performs well in a small grid-world can be unsuitable for high-dimensional observations. Likewise, DQN may be unnecessary when a small Q-table solves the problem efficiently. Therefore, experimental evaluation should report both performance and resource requirements rather than only a single accuracy-like number.

9. Conclusion

This research review analyzed Q-Learning, SARSA, and DQN as important reinforcement-learning methods. Q-Learning is an off-policy tabular method that learns toward the maximum

estimated future value, SARSA is an on-policy method that learns from the action actually selected, and DQN replaces the Q-table with a neural network and uses mechanisms such as replay memory and a target network for more stable learning. The comparison demonstrates a clear trade-off between simplicity, interpretability, scalability, and computational cost. The identified research gap concerns adaptive exploration and efficient experience selection. A feasible research direction is therefore to evaluate adaptive epsilon control and performance-aware sampling against the three baseline algorithms under a common benchmark.

10. Implementation Methodology

10.1 Experimental Environment

- Benchmark environment: CartPole-v1.
- Algorithms: tabular Q-Learning, tabular SARSA, and DQN.
- Evaluation budget: 500 training episodes per algorithm.
- Maximum steps per episode: 500.
- Primary metric: average episodic reward.
- Secondary metrics: final-100-episode average, convergence speed, and training time.
- For stronger research validity, repeat the experiment with at least five random seeds.

10.2 Expected Output

- Training reward curve for Q-Learning.
- Training reward curve for SARSA.
- Training reward curve for DQN.
- Combined performance chart.
- Final average reward comparison.
- Optional moving-average reward and convergence episode.

10.3 Interpretation of Results

Do not invent numerical results before running the code. The final report should insert the actual values produced by the implementation. For example, report: 'Q-Learning achieved a final-100-episode mean reward of X', 'SARSA achieved Y', and 'DQN achieved Z'. Then explain why the observed result may have occurred in terms of representation, exploration, convergence, and computational cost.

11. Implementation Code

The following Python implementation compares Q-Learning, SARSA, and DQN in CartPole-v1. Install the required packages first with: `pip install gymnasium torch numpy matplotlib`

```
import random

import time

import numpy as np

import matplotlib.pyplot as plt

import gymnasium as gym

import torch

import torch.nn as nn

import torch.optim as optim

from collections import deque

SEED = 42

EPISODES = 500

MAX_STEPS = 500

GAMMA = 0.99

ALPHA = 0.10

EPS_START = 1.00

EPS_MIN = 0.01

EPS_DECAY = 0.995

def seed_everything(seed=SEED):

    random.seed(seed)

    np.random.seed(seed)

    torch.manual_seed(seed)

# ----- Discretization for tabular Q-Learning and SARSA -----

LOW = np.array([-4.8, -4.0, -0.418, -4.0], dtype=np.float32)

HIGH = np.array([4.8, 4.0, 0.418, 4.0], dtype=np.float32)

BINS = np.array([20, 20, 20, 20])
```

```

def discretize(state):
    state = np.clip(state, LOW, HIGH)
    ratios = (state - LOW) / (HIGH - LOW)
    idx = (ratios * BINS).astype(int)
    idx = np.minimum(idx, BINS - 1)
    return tuple(idx)

def epsilon_greedy(q_values, epsilon):
    if random.random() < epsilon:
        return random.randrange(len(q_values))
    return int(np.argmax(q_values))

# ----- Q-Learning -----

def train_q_learning():
    env = gym.make("CartPole-v1")
    q_table = np.zeros((*BINS, env.action_space.n), dtype=np.float32)
    rewards = []
    epsilon = EPS_START
    for ep in range(EPISODES):
        state, _ = env.reset(seed=SEED + ep)
        s = discretize(state)
        total_reward = 0
        for _ in range(MAX_STEPS):
            action = epsilon_greedy(q_table[s], epsilon)
            next_state, reward, terminated, truncated, _ = env.step(action)
            ns = discretize(next_state)
            target = reward
            if not (terminated or truncated):
                target += GAMMA * np.max(q_table[ns])

```

```

        q_table[s + (action,)] += ALPHA * (
            target - q_table[s + (action,)]
        )

    s = ns

    total_reward += reward

    if terminated or truncated:
        break

    rewards.append(total_reward)

    epsilon = max(EPS_MIN, epsilon * EPS_DECAY)

env.close()

return rewards, q_table

# ----- SARSA -----

def train_sarsa():

    env = gym.make("CartPole-v1")

    q_table = np.zeros((*BINS, env.action_space.n), dtype=np.float32)

    rewards = []

    epsilon = EPS_START

    for ep in range(EPISODES):

        state, _ = env.reset(seed=1000 + ep)

        s = discretize(state)

        action = epsilon_greedy(q_table[s], epsilon)

        total_reward = 0

        for _ in range(MAX_STEPS):

            next_state, reward, terminated, truncated, _ = env.step(action)

            ns = discretize(next_state)

            total_reward += reward

            if terminated or truncated:

                target = reward

```

```

        q_table[s + (action,)] += ALPHA * (
            target - q_table[s + (action,)]
        )
        break

    next_action = epsilon_greedy(q_table[ns], epsilon)
    target = reward + GAMMA * q_table[ns + (next_action,)]
    q_table[s + (action,)] += ALPHA * (
        target - q_table[s + (action,)]
    )

    s = ns
    action = next_action

    rewards.append(total_reward)

    epsilon = max(EPS_MIN, epsilon * EPS_DECAY)

env.close()

return rewards, q_table

# ----- DQN -----

class DQN(nn.Module):

    def __init__(self, state_dim, action_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(state_dim, 128),
            nn.ReLU(),
            nn.Linear(128, 128),
            nn.ReLU(),
            nn.Linear(128, action_dim)
        )

    def forward(self, x):
        return self.net(x)

```

```

def train_dqn():

    env = gym.make("CartPole-v1")

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

    state_dim = env.observation_space.shape[0]

    action_dim = env.action_space.n

    policy_net = DQN(state_dim, action_dim).to(device)
    target_net = DQN(state_dim, action_dim).to(device)

    target_net.load_state_dict(policy_net.state_dict())

    target_net.eval()

    optimizer = optim.Adam(policy_net.parameters(), lr=0.001)

    replay = deque(maxlen=50000)

    batch_size = 64

    target_update = 10

    epsilon = 1.0

    epsilon_min = 0.05

    epsilon_decay = 0.995

    rewards = []

    def select_action(state, eps):

        if random.random() < eps:

            return env.action_space.sample()

        with torch.no_grad():

            s = torch.tensor(state, dtype=torch.float32,

                             device=device).unsqueeze(0)

            return int(policy_net(s).argmax(dim=1).item())

    for ep in range(EPISODES):

        state, _ = env.reset(seed=2000 + ep)

        total_reward = 0

        for _ in range(MAX_STEPS):

```

```

action = select_action(state, epsilon)

next_state, reward, terminated, truncated, _ = env.step(action)

done = terminated or truncated

replay.append((state, action, reward, next_state, done))

state = next_state

total_reward += reward

if len(replay) >= batch_size:

    batch = random.sample(replay, batch_size)

    states, actions, rewards_b, next_states, dones = zip(*batch)

    states_t = torch.tensor(np.array(states), dtype=torch.float32,
                           device=device)

    actions_t = torch.tensor(actions, dtype=torch.int64,
                           device=device).unsqueeze(1)

    rewards_t = torch.tensor(rewards_b, dtype=torch.float32,
                           device=device)

    next_states_t = torch.tensor(np.array(next_states),
                              dtype=torch.float32,
                              device=device)

    dones_t = torch.tensor(dones, dtype=torch.float32,
                          device=device)

    current_q = policy_net(states_t).gather(1, actions_t).squeeze(1)

    with torch.no_grad():

        next_q = target_net(next_states_t).max(dim=1).values

        target_q = rewards_t + GAMMA * next_q * (1 - dones_t)

    loss = nn.functional.smooth_l1_loss(current_q, target_q)

    optimizer.zero_grad()

    loss.backward()

    optimizer.step()

```

```

        if done:
            break

    rewards.append(total_reward)

    epsilon = max(epsilon_min, epsilon * epsilon_decay)

    if (ep + 1) % target_update == 0:
        target_net.load_state_dict(policy_net.state_dict())

env.close()

return rewards, policy_net

# ----- Evaluation -----

def moving_average(values, window=20):
    values = np.asarray(values, dtype=np.float32)

    if len(values) < window:
        return values

    return np.convolve(values, np.ones(window)/window, mode="valid")

def summarize(name, rewards):
    last100 = np.mean(rewards[-100:])
    best = np.max(rewards)

    print(f"{name}")
    print(f" Final 100-episode mean: {last100:.2f}")
    print(f" Best episode reward: {best:.2f}")
    print(f" Total episodes: {len(rewards)}")
    print()

if __name__ == "__main__":
    seed_everything()

    start = time.time()

    q_rewards, q_table = train_q_learning()

    q_time = time.time() - start

    start = time.time()

```

```

sarsa_rewards, sarsa_table = train_sarsa()

sarsa_time = time.time() - start

start = time.time()

dqn_rewards, dqn_model = train_dqn()

dqn_time = time.time() - start

summarize("Q-Learning", q_rewards)

summarize("SARSA", sarsa_rewards)

summarize("DQN", dqn_rewards)

print(f"Q-Learning time: {q_time:.2f} seconds")

print(f"SARSA time:      {sarsa_time:.2f} seconds")

print(f"DQN time:       {dqn_time:.2f} seconds")

plt.figure(figsize=(10, 6))

plt.plot(moving_average(q_rewards), label="Q-Learning")

plt.plot(moving_average(sarsa_rewards), label="SARSA")

plt.plot(moving_average(dqn_rewards), label="DQN")

plt.xlabel("Episode")

plt.ylabel("Moving Average Reward")

plt.title("Q-Learning vs SARSA vs DQN on CartPole-v1")

plt.legend()

plt.grid(True)

plt.tight_layout()

plt.show()

```

12. Result Recording Format

Run the implementation and replace the blank cells below with the actual values. Do not fabricate results.

Metric	Q-Learning	SARSA	DQN
Final 100-episode mean reward	_____	_____	_____
Best episode reward	_____	_____	_____

Training time (seconds)	_____	_____	_____
Convergence episode			
Memory requirement	Low	Low	High
Interpretability	High	High	Lower

References

1. Mnih, V., Kavukcuoglu, K., Silver, D., et al. (2015). Human-level control through deep reinforcement learning. *Nature*, 518, 529–533.
2. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.
3. Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. *Machine Learning*, 8, 279–292.
4. Rummery, G. A., & Niranjan, M. (1994). On-line Q-learning using connectionist systems. University of Cambridge Technical Report.
5. Improved SARSA and DQN algorithms for reinforcement learning. *Theoretical Computer Science*, 1027 (2025), 115025. [□cite□turn0search0□](#)
6. Online RL-based cloud autoscaling for scientific workflows: Evaluation of Q-Learning and SARSA. *Future Generation Computer Systems* (2024). [□cite□turn0search6□](#)
7. A review of reinforcement learning: A tripartite framework of environment design, algorithmic innovation, and application scenarios. *Array*, 30 (2026), 100812. [□cite□turn0search1□](#)
8. Synthesis of model predictive control and reinforcement learning: Survey and classification. *Annual Reviews in Control*, 61 (2026), 101045. [□cite□turn0search3□](#).